

Master 1 Info
Valeur Logiques
Partie logique du premier ordre
Cours 3

Jean-Marie Le Bars

Ensembles récursivement énumérables

Ensemble dénombrable

Un ensemble E est dénombrable lorsque l'on peut numéroter les éléments de E ,

$$E = \{e_0, e_1, e_2, \dots\}.$$

Ensemble récursivement énumérable

Un ensemble E est dit récursivement énumérable lorsqu'il existe un programme qui énumère tous ses éléments de sorte que tout élément de E est obtenu au bout d'un nombre fini d'étapes.

Notons $Enum(E)$ une telle énumération. Le rang d'un élément $r(e)$ est le nombre d'étapes que $Enum(e)$ met à produire cet élément.

Lien avec un ensemble dénombrable

Un ensemble récursivement énumérable est dénombrable.

- 1 on commence par numéroter les éléments de E ,

$$E = \{e_0, e_1, e_2, \dots\}.$$

- 2 $Enum(E)$ énumère (ou génère) les éléments de E un par un
 e_0 , puis e_1 , puis $e_2 \dots$

Exemple – $\mathbb{Z}$

On énumère $\mathbb{Z}$ avec l'ordre des éléments

$$\{0, 1, -1, 2, -2, 3, -3, \dots\}.$$

Mais commencer par énumérer les entiers relatifs positifs, puis les entiers relatifs négatifs ne marche pas :

$$\{0, 1, 2, \dots\} \text{ puis } \{-1, -2, -3, \dots\}.$$

En effet, les entiers négatifs ne seront pas obtenus au bout d'un nombre fini d'étapes.

Différence entre ensemble dénombrable et ensemble récursivement énumérable

- si E est dénombrable il existe une numérotation
- nous ne sommes pas forcément en mesure de trouver cette numérotation
- pour $\mathbb{Z}$ $Enum(\mathbb{Z})$ était facile à trouver

Fonction calculable

Définition

Une fonction est calculable lorsqu'il existe un programme qui, pour tout argument donné, fournit la valeur de la fonction pour cet argument en un nombre fini d'étapes.

Fonction calculable et ensemble récursivement énumérable

Une fonction calculable peut calculer le rang des éléments d'un ensemble.

- on dispose d'un ensemble E récursivement énumérable, soit $\text{Enum}(E)$ une énumération.
- la fonction calculable est une fonction $f : E \rightarrow \mathbb{N}$, avec

$$f(e) = r(e),$$

où $r(e)$ est le rang de e dans Enum .

Exemple avec $\mathbb{Z}$

On définit la fonction $f : \mathbb{Z} \rightarrow \mathbb{N}$ par

$$\begin{aligned} f(0) &= 0 \\ f(i) &= 2i - 1, \text{ pour } i \text{ strictement positif} \\ f(i) &= -2i, \text{ pour } i \text{ strictement négatif} \end{aligned}$$

f est clairement calculable, ce qui prouve que $\mathbb{Z}$ est récursivement énumérable.

$\mathbb{N}^k$ est dénombrable

- pour tout $k \in \mathbb{N}$, $\mathbb{N}^k$ est un ensemble dénombrable.
- nous nous intéresserons aux fonctions calculables de la forme

$$f : \mathbb{N}^k \rightarrow \mathbb{N}$$

Comment définir les fonctions calculables

La notion de programme peut paraître flou, mais elle a été introduite simultanément par plusieurs logiciens avec des formalismes différents.

Récurtivité

Les fonctions calculables sont celles qui peuvent être définies par des **fonctions récursives**.

Machines de Turing

Les fonctions calculables sont celles qui peuvent être calculées par des **machines de Turing**.

Lambda calcul

Les fonctions calculables sont celles qui peuvent être définies par le **lambda calcul**.

Thèse de Church-Turing

Une thèse consiste à affirmer, sans le démontrer, qu'une notion intuitive correspond exactement à une notion formelle.

La thèse de Church-Turing (énoncé après les travaux de ces logiciens) énonce que ces trois approches répondent à la notion intuitive de calculabilité.

Fonctions récursives primitives

Les fonctions récursives primitives forment une classe importante de fonctions calculables

Fonctions de bases

- B1 On part de la **structure** $(\mathbb{N}, 0, S)$
- B2 Pour tout $k \in \mathbb{N}$, nous avons la fonction d'arité k , $Nul_k(t_1, \dots, t_k)$ qui, pour toute suite de k termes $t_1, \dots, t_k$, donne 0.

$$Nul_k(t_1, \dots, t_k) = 0.$$

- B3 **Projection de la i ème composante**

$$Proj_i(t_1, \dots, t_k) = t_i,$$

pour tout $i \in \{1, \dots, k\}$.

Règles de composition

- R1 **Règle de composition :**

Soient $g(t_1, \dots, t_m)$, $h_1(t_1, \dots, t_n), \dots, h_m(t_1, \dots, t_n)$ des fonctions récursives primitives.

Alors la fonction $f(t_1, \dots, t_n)$ suivante est également récursive primitive

$$f(t_1, \dots, t_n) = g(h_1(t_1, \dots, t_n), \dots, h_m(t_1, \dots, t_n)).$$

- R2 **Règle de récursion (ou induction) :**

Soient $g(t_1, t_2)$ une fonction récursive primitive et $f(t_1, \dots, t_{n+1})$ une fonction pour laquelle nous savons calculer $f(t_1, \dots, t_n, 0) = n_0$.

Pour les autres valeurs prises par t_{n+1} , nous avons la relation

$$f(t_1, \dots, t_n, S(k)) = g(k, f(t_1, \dots, t_n, k)).$$

Par exemple, si $f(t_1, \dots, t_n) = 3$, il vient $f(t_1, \dots, t_n) = g(6, g(5, g(4, g(3, g(2, g(1, g(0, 3))))))$.

Alors cette fonction $f(t_1, \dots, t_n)$ est également récursive primitive.

Fonctions récursives primitives

Une fonction récursive même très simple peut ne pas être récursive primitive.

L'exemple le plus connu est celui de la fonction d'Ackermann.

Fonction d'Ackermann

La fonction d'Ackermann A est une fonction de $\mathbb{N}^2$ vers $\mathbb{N}$ définie par

$$\begin{aligned} A(0, k) &= S(k) \\ A(S(k), 0) &= A(k, S(0)) \\ A(S(k_1), S(k_2)) &= A(k_1, A(S(k_1), k_2)) \end{aligned}$$

Cette fonction se programme très simplement.

Idée de la preuve

La troisième règle de construction

$$A(S(k_1), S(k_2)) = A(k_1, A(S(k_1), k_2))$$

fait intervenir **une récurrence sur les deux arguments** alors que la règle de récursion des fonctions récursives primitives a une récurrence qui ne porte que sur un argument.

$$f(t_1, \dots, t_n, S(k)) = g(k, f(t_1, \dots, t_n, k)).$$

Décidabilité d'une propriété sur une logique

Propriété sur une logique

Soit $\mathcal{L}$ une logique.

Une propriété $\mathcal{P}$ sur $\mathcal{L}$ est un sous-ensemble de $\mathcal{L}$.

Exemples de propriétés

- $\text{SAT}(\mathcal{L})$ est l'ensemble des énoncés de $\mathcal{L}$ qui sont satisfaisables.
- $\text{INSAT}(\mathcal{L})$ est l'ensemble des énoncés de $\mathcal{L}$ qui sont contradictoires (antilogies).
- $\text{VAL}(\mathcal{L})$ est l'ensemble des énoncés de $\mathcal{L}$ qui sont valides (tautologies).
- $\text{NONVAL}(\mathcal{L})$ est l'ensemble des énoncés de $\mathcal{L}$ qui ne sont pas valides.

Propriété décidable

Soit $\mathcal{L}$ une logique et $\mathcal{P}$ une propriété sur $\mathcal{L}$.

$\mathcal{P}$ est décidable sur $\mathcal{L}$

lorsqu'il existe un programme qui, pour tout énoncé φ de $\mathcal{L}$, démontre au bout d'un nombre fini d'instructions une des deux assertions

- φ vérifie $\mathcal{P}$.
- φ ne vérifie pas $\mathcal{P}$.

Propriété semi-décidable

$\mathcal{P}$ est semi-décidable lorsque les énoncés de $\mathcal{L}$ vérifiant $\mathcal{P}$ forment un ensemble récursivement énumérable.

Lien entre décidabilité et semi-décidabilité

Logique close par négation

Une logique $\mathcal{L}$ est **close par négation** lorsque, pour tout énoncé φ , nous avons l'équivalence

$$\varphi \in \mathcal{L} \iff \neg\varphi \in \mathcal{L}.$$

Exemples de logiques closes par négation

FO et SO sont closes par négation.

Lien entre satisfaisabilité et validité

Soit $\mathcal{L}$ une logique close par négation.

Nous avons l'équivalence

- $\text{SAT}(\mathcal{L})$ est décidable sur $\mathcal{L}$
- $\text{VAL}(\mathcal{L})$ est décidable sur $\mathcal{L}$

Equivalence

Supposons que $\mathcal{L}$ soit close par négation.

Nous avons l'équivalence

- $\mathcal{P}$ est décidable sur $\mathcal{L}$
- $\mathcal{P}$ et $\text{non}\mathcal{P}$ sont semi-décidables sur $\mathcal{L}$

Idée de la preuve

Soit E_1 une énumération des éléments de $\mathcal{P}$ et E_2 une énumération des éléments de $\neg\mathcal{P}$.

- On énumère alternativement les éléments de E_1 et E_2
- Si φ vérifie $\mathcal{P}$ et E_1 l'énumère en k_1 étapes, nous obtenons φ en $2k_1 - 1$ étapes avec cette nouvelle énumération
- φ vérifie $\text{non}\mathcal{P}$ et E_2 l'énumère en k_2 étapes, nous obtenons φ en $2k_2$ étapes avec cette nouvelle énumération

Indécidabilité de FO

Pour montrer l'indécidabilité de $\text{val}(\text{FO})$, on réduit un problème déjà connu comme indécidable à $\text{val}(\text{FO})$.

Problème POST

Le problème de Post est un problème de décision. On se fixe un alphabet. On construit des dominos sur chacun desquels on écrit deux mots. Une instance du problème est ainsi une suite finie

$$(u_1, v_1), (u_2, v_2), \dots, (u_n, v_n)$$

de couples de mots.

Le problème consiste à savoir si on peut aligner des dominos de sorte que les mots résultant sur chaque moitié sont les mêmes :

Existe-t-il une suite $k_1, k_2, \dots, k_p$ d'indices à prendre dans $[1, n]$ (on peut choisir plusieurs fois le même indice) telle que les mots

$$u_{k_1} u_{k_2}, \dots, u_{k_p} \text{ et } v_{k_1}, v_{k_2}, \dots, v_{k_p}$$

soient identiques.

Réduction de POST à VAL(FO)

- on part d'une instance W de POST
- on construit un énoncé φ de FO
- on montre que φ est valide si et seulement si nous avons une solution pour POST sur W .

Satisfaisabilité

On en déduit que $\text{SAT}(\text{FO})$ est indécidable.

Classes préfixes de la logique du premier ordre

SAT(FO) et VAL(FO) sont indécidables, mais nous pouvons obtenir la décidabilité pour des fragments de FO.

Forme prénexe

Un énoncé de FO est sous forme prénexe lorsque tous les quantificateurs se trouvent au début de l'énoncé.

Exemple

$$\forall x \exists y \forall z \left(z = y \vee (E(x, y) \wedge \neg E(x, z)) \right. \\ \left. \vee (\neg E(x, y) \wedge E(x, z)) \right)$$

Préfixe

Le préfixe d'un énoncé sous forme prénexe est défini par les alternances des quantificateurs.

Un préfixe P est un mot sur l'alphabet $\{\forall, \exists\}$.

Exercice

Mettez sous forme prénexe de préfixe $\forall\exists\exists$ l'énoncé suivant (énoncé φ_1 du TP 1).

$$\forall x \left(\left(\exists y E(x, y) \right) \wedge \left(\exists y \neg E(x, y) \right) \right).$$

Exercice – Diamètre 2

Mettez sous forme prénexe de préfixe $\forall\forall\exists$ l'énoncé suivant exprimant que le graphe est de diamètre 2 (énoncé φ_3 du TP 1 un peu modifié).

$$\forall x \forall y \left(x = y \vee E(x, y) \vee (\exists z E(x, z) \wedge E(z, y)) \right).$$

Classes préfixes de la logique du premier ordre

Langages rationnels sur $\{\exists, \forall\}$.

- Une classe préfixe est définie à partir d'un langage rationnel $\mathcal{L}$ sur $\{\exists, \forall\}$.
- $\forall^*$ signifie que nous avons une suite de $\forall$ (qui peut être vide)
- $\exists^*$ signifie que nous avons une suite de $\exists$ (qui peut être vide)

Classe préfixe

Une classe préfixe sur $\mathcal{L}$ est l'ensemble des énoncés sous forme préfixe de préfixe P où P est un mot de $\mathcal{L}$.

Principales classes préfixes

Certaines des classes préfixes portent le nom d'un ou plusieurs logiciens qui ont travaillé sur ces classes.

- | | |
|-----------------------|-------------------------------|
| ● Ackermann | $\exists^* \forall \exists^*$ |
| ● Bernays-Schönfinkel | $\exists^* \forall^*$ |
| ● Gödel | $\exists^* \forall \exists^*$ |
| ● Gödel minimale | $\forall \forall \exists$ |
| ● Kahr-Moore-Wang | $\forall \exists \forall$ |

Pouvoir d'expression d'une classe préfixe

Nous pouvons rechercher

- les propriétés qui sont définissables dans la classe préfixe
- les propriétés qui ne sont pas définissables dans la classe préfixe

Classes préfixes de la logique du premier ordre

Skolémisation

- la skolémisation d'un énoncé sous forme prénexe consiste à
supprimer tous les quantificateurs $\exists$

- on obtient un préfixe $\forall^k$, pour un certain $k \in \mathbb{N}$

- changement de vocabulaire :

cette suppression s'effectue en ajoutant au vocabulaire des fonctions, des constantes ou des relations.

Exemple 1 de skolémisation

On part de l'énoncé

$$\varphi_1 = \exists x \exists y \forall z \phi(x, y, z),$$

où ϕ a comme variables libres x, y et z . On remplace φ_1 par

$$\varphi_1^S = \forall z \phi(a, b, z),$$

où a et b sont des constantes ajoutées au vocabulaire.

Exemple 2 de Skolémisation

On part de l'énoncé

$$\varphi_2 = \forall x \exists y \exists z R(x, y) \wedge S(x, z),$$

On remplace φ_2 par

$$\varphi_2^S = \forall x R(x, f_1(x)) \wedge S(x, f_2(x)),$$

où f_1 et f_2 sont deux fonctions unaires d'arité 1 appelées fonctions de Skolem ajoutées au vocabulaire.

Classes préfixes décidables et classes préfixes indécidables

Nous considérerons toujours $\text{SAT}(\mathcal{L})$ pour les résultats qui suivent.

Classes préfixes décidables

$\text{SAT}(P)$ est décidable pour les préfixes P suivants :

- La classe d'Ackermann $\exists^* \forall \exists^*$ (Ackerman, 1928)
- La classe de Bernays-Schönfinkel $\exists^* \forall^*$ (Bernays-Schönfinkel, 1928)

Fragments de FO à k variables

Soit $k \in \mathbb{N}^*$, nous noterons FO^k l'ensemble des énoncés de FO comportant au plus k variables.

Il est évident que toutes ces logiques sont closes par négation.

Classes préfixes indécidables

$\text{SAT}(P)$ est indécidable pour les préfixes P suivants :

- La classe de Kahr-Moore-Wang $\forall \exists \forall$ (Kahr-Moore-Wang, 1963)
On code un problème de dominos.
- La classe de Gödel minimale $\forall \forall \exists$ (Goldfarb, 1991)
On code un axiome de l'infini que l'on utilise pour faire une réduction à un problème indécidable.

Décidabilité de FO^2

FO^2 est décidable (Mortimer 1975)

Indécidabilité de FO^k

FO^k est indécidable, pour $k \geq 3$.

Preuve : FO^3 contient la classe de Kahr-Moore-Wang et contient la classe de Gödel minimale.